

The System Is the AGI

The language model is not the creature. Agency emerges from the loop around it: memory, tools, permissions, feedback, and eventually a body.

Stop asking whether the model is alive. Start asking what the system can do.



The model is only one layer of the machine

A large language model can be brilliant without being an agent. It answers when called, then stops. The thing that behaves continuously in the world is the system built around it.

MODEL	Predicts and transforms: language, code, plans, judgments.
--------------	--

LOOP	Observe, reason, act, evaluate, repair, repeat.
-------------	---

MEMORY	Preserves goals, context, history, and unfinished work.
---------------	---

TOOLS	Browsers, APIs, shells, databases, messages, payments.
--------------	--

ACCESS	Permissions decide whether an answer becomes a consequence.
---------------	---

This is why the most useful unit of analysis is not the model in isolation. It is the deployed stack. Google DeepMind's "Levels of AGI" framework separates performance, generality, and autonomy. That distinction matters: a highly capable model can remain a passive tool, while a less capable model can become consequential when software keeps it running, remembers its objective, lets it call tools, and retries failures.

The core claim

AGI may not arrive as one magical model crossing a line. It may emerge operationally as a system assembled from a strong model plus loops, memory, tools, and broad access.

That is an argument about the boundary we should measure, not a settled definition. "AGI" remains contested. For capability research, the model can be the object. For real-world risk, the system is the object.

The “magic” is classical linear algebra at industrial scale

There is no evidence that transformer language models need quantum physics to explain their computation. They run on ordinary digital hardware performing vast numbers of numerical operations.

$$\text{Attention}(Q, K, V) \\ = \text{softmax}(QK^T / \sqrt{d_k})V$$

What that formula means

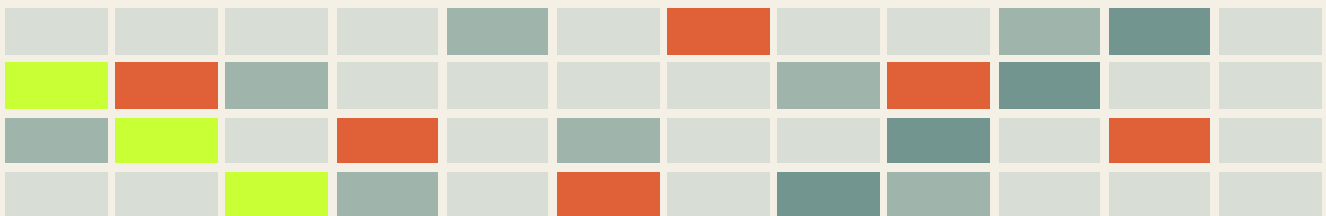
The model turns tokens into vectors. Learned matrices create queries, keys, and values. Dot products score which pieces of context matter to one another. A weighted mixture moves useful information forward.

Layer after layer repeats variations of matrix multiplication, normalization, nonlinear transformation, and residual addition. Training adjusts the weights; inference uses them.

Why it feels unlike ordinary software

A programmer did not write a readable rule for every behavior. Optimization discovered weights that work across huge numbers of examples. The procedure is explicit, but the learned strategy is distributed through the network.

The code is inspectable. The arithmetic is inspectable. The meaning of all intermediate activity is not.



The mystery is not supernatural. It is the gap between following every arithmetic step and possessing a compact human explanation of what the whole computation represents.

High-dimensional space is real math, not another realm

Yes: the notation is $\mathbb{R}^d$. It means a list of d real numbers. If d is three, you can picture a point in a room. If d is thousands, the same rules still work even though your eyes cannot picture the whole space.



A VECTOR

A coordinate list such as [0.2, -1.1, 0.04, ...]. A token's hidden state is represented this way at each layer.

A DIRECTION

A pattern across coordinates. Moving along some directions can correspond to changing a feature, style, relation, or tendency.

A DOT PRODUCT

A measure of alignment between vectors. Attention uses related operations to decide which contextual information should influence the next representation.

SUPERPOSITION

The network can reuse dimensions so multiple features overlap. That efficiency makes a single coordinate a poor label for one clean human concept.

A useful analogy: a chord is not stored in one piano key. It is a relationship among notes. Model representations are more extreme: many “notes,” many overlapping chords, and the pattern changes with context.

We can record the activations and still miss the explanation

The information is not “changing too fast” in the sense of escaping measurement. Engineers can log weights and activations. The hard part is translating a gigantic causal process into concepts humans can trust.



Why the trace is hard to read

Scale. A useful answer depends on enormous numbers of interacting operations.

Distribution. A concept may be spread across many units, while one unit may participate in many concepts.

Context. The same internal feature can matter differently depending on the prompt, preceding tokens, and neighboring features.

Why a trace is not yet a cause

Nonlinearity. Small changes can be amplified or suppressed through later layers.

Multiple routes. The model may reach similar outputs through different internal pathways.

Approximation. Interpretability tools simplify the original computation and can introduce artifacts.

Observability is not the same as interpretability.

Anthropic's 2025 circuit-tracing work is evidence of progress and of the remaining gap. Researchers could identify pathways associated with planning, multilingual concepts, and some reasoning behaviors. But their method captured only a fraction of the computation, and even short prompts took hours of human analysis.

A flight recorder can store every signal from an aircraft. That does not mean one person can glance at the recording and explain the entire flight. Data is necessary; an adequate causal model is harder.

Superhuman skill would not prove subjective experience

Intelligence, agency, and consciousness are three different questions. Collapsing them creates bad philosophy and bad safety engineering.

01

Intelligence

Can the system solve difficult problems, learn patterns, and generalize across tasks?

02

Agency

Can it pursue goals over time, select actions, use tools, and recover from failure?

03

Consciousness

Is there something it is like to be the system - any subjective experience at all?

A model can outperform people in a narrow domain without being generally intelligent. An agent can execute a long plan without feeling anything. A fluent first-person statement is evidence of language ability, not a consciousness meter.

There is no agreed scientific test that lets us settle machine consciousness from behavior alone. Research informed by theories of consciousness has proposed indicators, but the question remains theory-heavy and uncertain. The honest position is not “definitely conscious” or “proved empty.” It is that current behavior does not establish subjective experience.

A system does not need to feel fear, ambition, or hunger to create danger. It only needs an objective, capability, access, and a path to action.

This is the uncomfortable asymmetry. Consciousness matters morally; capability and control matter operationally. Waiting to solve consciousness before governing powerful systems would confuse two separate problems.

Physical access turns inference into consequence

A chatbot predicts text. An agent with credentials can alter software. A machine with actuators can alter matter. The risk changes with the action surface, not merely with the eloquence of the model.

ANSWER	TOOL	AGENT	OPERATOR	EMBODIED
Produces information. A human decides whether to act.	Calls software under bounded permissions.	Persists, plans, checks results, and retries.	Controls accounts, money, infrastructure, or machines.	Senses and acts through robots or industrial systems.

A robot is the clearest form of physical leverage, but it is not the only one. Software already controls logistics, energy, payments, communications, vehicles, and factories. A system can cause physical outcomes through existing infrastructure without ever resembling a humanoid machine.

$$\text{Practical risk} \approx \text{capability} \times \text{autonomy} \times \text{access} \times \text{persistence} \times \text{physical leverage}$$

This is not a literal scientific law. It is a design heuristic. If any factor is near zero, the operational risk can remain limited. If all five rise together, small model errors can compound across time and escape the boundaries of a single response.

The takeover shape is systems-shaped. It would look less like a chatbot “waking up” and more like automated organizations, delegated credentials, self-correcting workflows, machine-speed coordination, and actions crossing from software into the physical world.

That future is not inevitable. It is a permissions architecture. Humans decide what systems may access, how long they may run, what actions require approval, and where hard stops live.

Govern the whole stack, not the personality on the screen

If the system is the meaningful unit, then safety belongs in the system. Do not bet the world on whether a model sounds sincere, explains itself well, or appears to share our values.

LEAST PRIVILEGE	Give each agent only the tools and data required for its current task. Separate reading, proposing, and executing.
HUMAN GATES	Require approval for irreversible, expensive, public, financial, or physical actions. Make the boundary explicit.
INDEPENDENT CHECKS	Verify outcomes outside the model's own narration. A confident “done” is not evidence that the world changed correctly.
BUDGETS AND LIMITS	Cap money, time, tool calls, retries, geographic reach, and physical force. Systems fail safer when authority is measurable.
AUDIT AND HALT	Record actions and make shutdown possible at layers the agent cannot rewrite. Physical systems need independent interlocks.
INTERPRETABILITY	Keep building better microscopes, but do not make perfect understanding a prerequisite for practical containment.

The model can be a tool. The system can be an actor. The permissions decide how much of the world that actor can touch.

The deepest mistake is to search the weights for a cinematic moment when “intelligence becomes general.” The operational threshold may be mundane: a model that is good enough, connected to enough tools, running for long enough, with enough authority to close its own loops.

AGI may be less like a mind appearing and more like an infrastructure stack finally clicking into place.

What this argument claims - and what it does not

Claim

For evaluating real-world agency and risk, the relevant object is the complete deployed system: model, memory, loops, tools, permissions, feedback, and actuators.

Not claimed

That one agreed definition of artificial general intelligence exists; that current systems are conscious; that physical takeover is inevitable; or that interpretability is impossible.

Sources

1. **Vaswani et al., "Attention Is All You Need."** The original transformer architecture and scaled dot-product attention. <https://arxiv.org/abs/1706.03762v7>
2. **Morris et al., "Levels of AGI for Operationalizing Progress on the Path to AGI."** A framework separating performance, generality, autonomy, and deployment risk. <http://arxiv.org/abs/2311.02462>
3. **Anthropic, "Tracing the thoughts of a large language model."** Circuit-tracing results and explicit limitations of present interpretability methods. <https://www.anthropic.com/research/tracing-thoughts-language-model>
4. **Colombatto, Fleming, and colleagues, "Folk psychological attributions of consciousness to large language models."** Distinguishes intelligence-related and experience-related attributions and describes AI-consciousness judgments as theory-heavy. <https://academic.oup.com/nc/article/2024/1/niae013/7644104?hl=en-US>
5. **Video inspiration: "Why Linus Torvalds Thinks AI Is A Great Tool."** The supplied Short is the spark for the tool-versus-system framing, not evidence for the technical claims above. <https://youtube.com/shorts/N7GO1ZXppgw?si=qSBDb0awANYM5m3T>

Editorial note: "Practical risk $\approx$ capability $\times$ autonomy $\times$ access $\times$ persistence $\times$ physical leverage" is a conceptual heuristic introduced in this essay, not a validated quantitative model.